

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓 名：张序鸿

学 院：竺可桢学院

专 业：计算机科学与技术

学 号：3230103265

指导教师：杨莹春

2024 年 12 月 13 日

浙江大学实验报告

课程名称：____计算机组成与设计____ 实验类型：____综合____

实验项目名称：____流水线处理器集成____

学生姓名：____张序鸿____ 专业：____混合班____ 学号：3230103265

指导老师：____杨莹春、潘之杰____ 助教：____马致远____

实验地点：____东 4-509____ 实验日期：2024 年 12 月 11 日

一、实验目的和要求

1. 理解流水线 CPU 的基本原理和组织结构
2. 掌握五级流水线的工作过程和设计方法
3. 理解流水线 CPU 停机的原理
4. 设计流水线测试程序

二、实验内容和原理

内容：

任务一：集成设计流水线 CPU，在 Exp04 的基础上完成

- 利用五级流水线各级封装模块集成 CPU
- 替换 Exp04 的单周期 CPU 为本实验集成的五级流水线 CPU

任务二：设计流水线测试方案并完成测试

原理：

2.1 五级流水线 – 具体操作

五级流水线主要分为 IF（取指）、ID（译码）、EXE（执行）、MEM（内存读写）、WB（寄存器读写）：

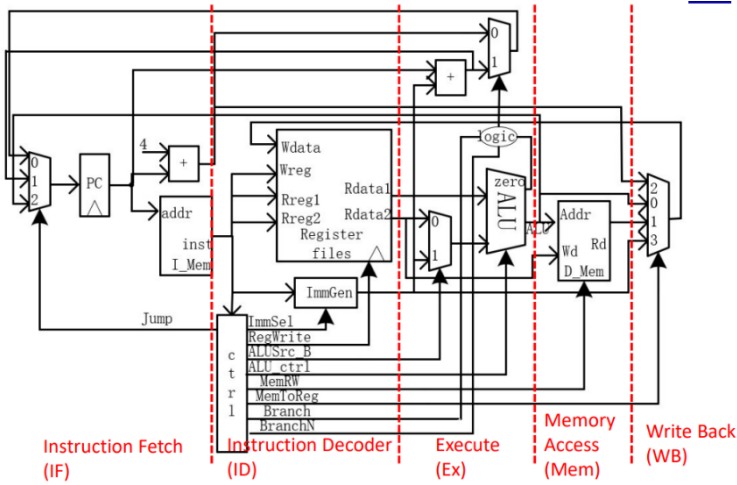
指令 \ 流水段	ALU	LOAD/STORE	BRANCH
IF	取指令	取指令	取指令
ID	译码，读寄存器文件	译码，读寄存器文件	译码，读寄存器文件
EXE	执行	计算访存有效地址	计算转移目标地址，设置条件码
MEM	(空操作)	对存储器读或写操作	若条件成立，将转移地址送 PC
WB	结果写入寄存器文件	读出数据写入寄存器文件	(空操作)

连续指令对应五级流水线时空图：

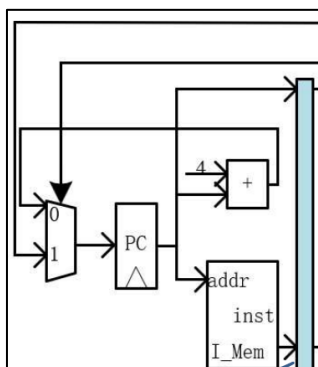
指令序列	流水时钟数								
	1	2	3	4	5	6	7	8	9
指令 i	IF	ID	EX	MEM	WB				
指令 i+1		IF	ID	EX	MEM	WB			
指令 i+2			IF	ID	EX	MEM	WB		
指令 i+3				IF	ID	EX	MEM	WB	
指令 i+4					IF	ID	EX	MEM	WB

2.2 五级流水线 Datapath

Pipeline 流水线构建的 Datapath 区别单周期 CPU 中的 Datapath 在于增加了若干寄存器（如 IF_reg_ID、ID_reg_EX、EX_reg_MEM、Mem_reg_WB），它们负责储存 5 个流水线上级的数据，从而实现每个时钟都能读取一条指令，并将上一级的数据传递到下一级。



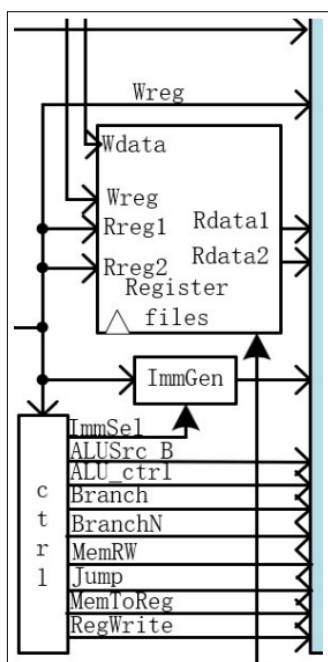
(1) Instruction Fetch (IF):



IF: 取指阶段涉及程序计数器 (PC) 和指令存储器 (I_Mem), 程序计数器输出作为地址从指令存储器中读取指令。

IF_reg_ID: 暂存指令、PC, 以待下一级使用。

(2) Instruction Decoder (ID):

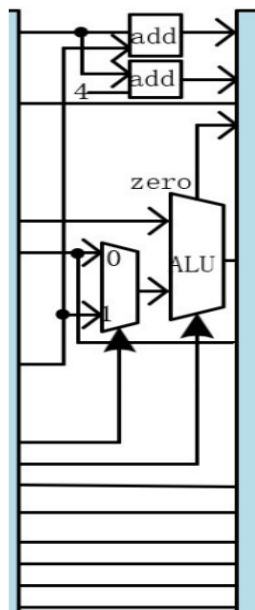


ID: 译码阶段涉及寄存器堆 (RegisterFiles) 和译码器 (SCPU_ctrl)、立即数生成单元 (ImmGen)。

从寄存器堆可以读取操作数后, 译码器对指令进行解析产生各种各种控制信号, 立即数生成单元根据控制信号和输入指令生成各种类型的立即数。

ID_reg_Ex: 暂存 PC 值, 寄存器读取的数据 (rs1, rs2), 立即数和控制信号以待下一级使用

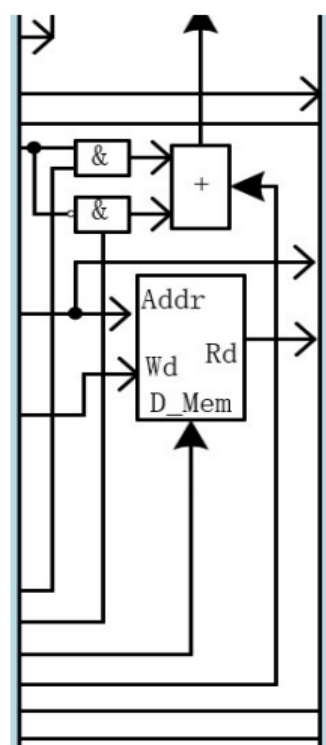
(3) Execute (Ex):



Ex : 执行阶段涉及运算单元 (ALU) 它获取操作数并完成指定的算数运算或逻辑运算。

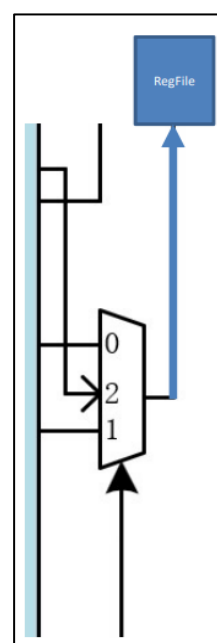
Ex_reg_Mem: 暂存运算结果和控制信号, 以待下一级使用。

(4) Memory Access (Mem):



Mem: 存储器访问阶段涉及数据存储器 (D_Mem), Load\Store 指令对数据存储器进行读或写。

Mem_reg_WB: 暂存存储器结果 和控制信号, 以待下一级使用



(5) Write Back (WB):

WB: 写回阶段涉及寄存器堆 (RegisterFiles), 将 ALU 的运算结果、存储器输出结果、PC+4 写回到寄存器堆。

此处不存在寄存器, 写回阶段结束, 一次完整的五级流水线操作完成!

2.3 五级流水线 SCPU_ctrl

流水线的控制信号同单周期 CPU 一样, 来自于对指令的译码操作输出。

不同点在于流水控制信号会根据每阶段的不同功能部件选择性控制输出, 本阶段用不到的信号会暂存于**寄存器**, 并逐级传递下去。

(1) IF 控制:

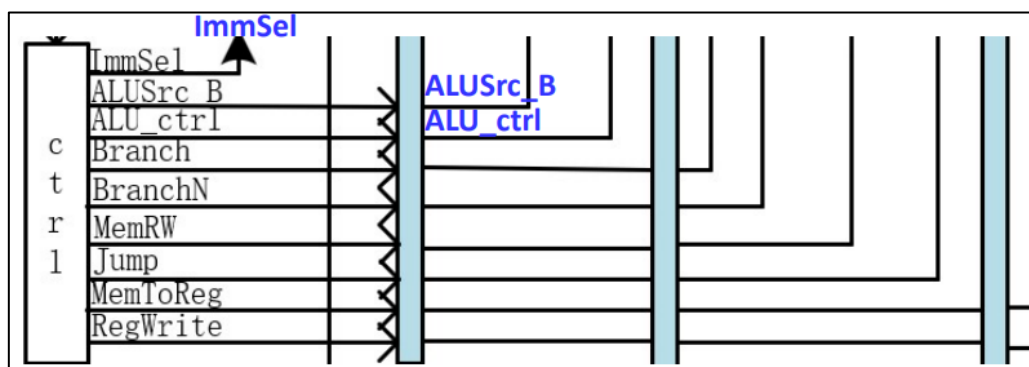
取指阶段, 读指令存储器和写 PC 值永远有效, 无需控制信号。

(2) ID 控制:

译码阶段，立即数生成单元需要根据指令类型产生对应输出，由控制信号 ImmSel 信号输出；其他信号暂存寄存器。

(3) Ex 控制:

执行阶段，ALU 的操作和第二个操作数 Src_B 需要选择，由控制信号 ALU_ctrl、ALUSrc_B 决定；其他信号暂存寄存器。

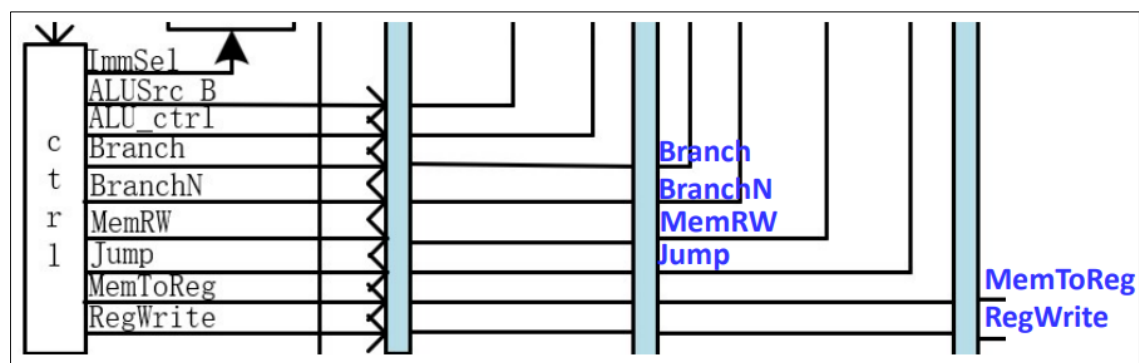


(4) Mem 控制:

存储器访问阶段，需要读写存储器以及根据分支 跳转指令判定选择 PC 转移值，MemRW、Branch、BranchN、Jump 输出；其他信号暂存寄存器。

(5) WB 控制:

写回阶段，ALU 运算结果、存储器输出等需要选择写回寄存器堆，同时寄存器堆的写使能需要设置；MemToReg、RegWrite 信号输出。

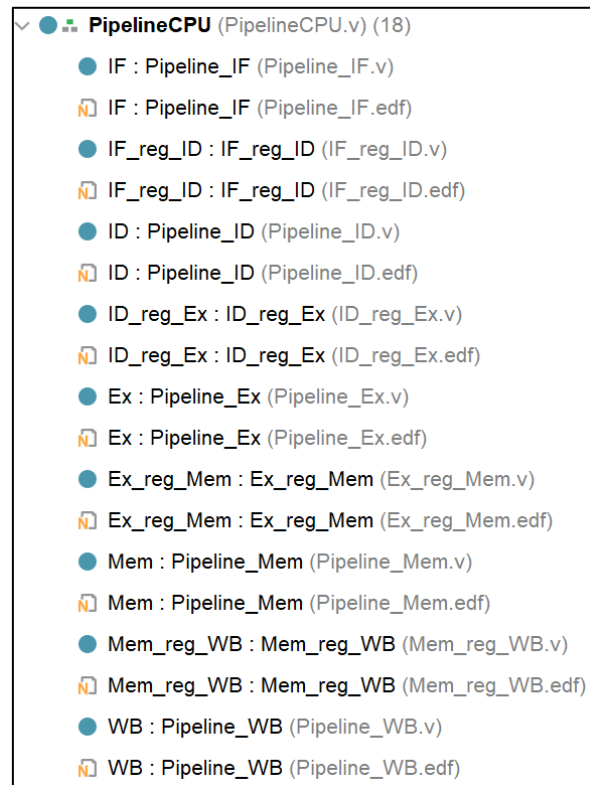


三、实验过程和数据记录

3.1 集成设计流水线 CPU:

创建 Pipeline_CPU.v 文件，将提供的流水线各阶段文件（.edf 文件）集成。

首先展示一下 Pipeline_CPU 的工程结构：



• PipelineCPU 接口设计：

```
module PipelineCPU(  
    input clk,           //时钟  
    input rst,           //复位  
    input[31:0] Data_in, //存储器数据输入  
    input[31:0] inst_IF,  //取指阶段指令  
    output [31:0] PC_out_IF, //取指阶段 PC 输出  
    output [31:0] PC_out_ID, //译码阶段 PC 输出  
    output [31:0] inst_ID, //译码阶段指令  
    output [31:0] PC_out_IDEX, //执行阶段 PC 输出  
    output [31:0] inst_EX, // 执行阶段指令  
    output MemRW_EX, //执行阶段存储器读写  
    output MemRW_Mem, //访存阶段存储器读写  
    output [31:0] Addr_out, //地址输出  
    output [31:0] Data_out, //CPU 数据输出  
    output [31:0] Data_out_WB, //写回数据输出  
    output [1023:0] all_regs_data  
);
```

- 流水线 CPU 执行过程中的传递信号:

```

•   wire [31:0] Rd_addr_out_ID , Rs1_out_ID , Rs2_out_ID,
    Imm_out_ID;
•   wire [3:0] ALU_control_ID;
•   wire [1:0] MemtoReg_ID , Jump_ID;
•   wire ALU_Src_B_ID , Branch_ID , BranchN_ID , MemRW_ID ,
    RegWrite_out_ID;
•
•   wire [31:0] PC_out_EX , Rs1_out_IDEX , Rs2_out_IDEX,
    Imm_out_IDEX;
•   wire [4:0] Rd_addr_out_IDEX;
•   wire [3:0] ALU_control_IDEX;
•   wire [1:0] MemtoReg_IDEX , Jump_IDEX;
•   wire ALUSrc_B_IDEX , Branch_IDEX , BranchN_IDEX ,
    MemRW_IDEX , RegWrite_IDEX;
•
•   wire [31:0] PC4_out_EX, ALU_out_EX, Rs2_out_EX;
•   wire zero_out_EX;
•
•   wire [31:0] PC_out_EXMem , PC4_out_EXMem ;
•   wire [4:0] Rd_addr_out_EXMem;
•   wire [1:0] MemtoReg_out_EXMem , Jump_out_EXMem;
•   wire zero_out_EXMem, Branch_out_EXMem, BranchN_out_EXMem,
    RegWrite_out_EXMem;
•
•   wire PCSrc;
•
•   wire [31:0] PC4_out_MemWB , ALU_out_MemWB,
    DMem_data_out_MemWB;
•   wire [4:0] Rd_addr_out_MemWB;
•   wire [1:0] MemtoReg_out_MemWB;
•   wire RegWrite_out_MemWB;

```

- 流水线 CPU 集成:

```

•   Pipeline_IF IF(
•       .clk_IF(clk),
•       .rst_IF(rst),
•       .en_IF(1'b1),
•       .PC_in_IF(PC_out_EXMem),
•       .PCSrc(PCSrc),
•       .PC_out_IF(PC_out_IF)
•   );
•   IF_reg_ID IF_reg_ID(

```



```

•     .clk_IFID(clk),
•     .rst_IFID(rst),
•     .en_IFID(1'b1),
•     .PC_in_IFID(PC_out_IF),
•     .inst_in_IFID(inst_IF),
•     .PC_out_IFID(PC_out_ID),
•     .inst_out_IFID(inst_ID)
• );
•     Pipeline_ID ID(
•     .clk_ID(clk),
•     .rst_ID(rst),
•     .RegWrite_in_ID(RegWrite_out_MemWB),
•     .Rd_addr_ID(Rd_addr_out_MemWB),
•     .Wt_data_ID(Data_out_WB),
•     .Inst_in_ID(inst_ID),
•     .Rd_addr_out_ID(Rd_addr_out_ID),
•     .Rs1_out_ID(Rs1_out_ID),
•     .Rs2_out_ID(Rs2_out_ID),
•     .Imm_out_ID(Imm_out_ID),
•     .ALUSrc_B_ID(ALUSrc_B_ID),
•     .ALU_Control_ID(ALU_control_ID),
•     .Branch_ID(Branch_ID),
•     .BranchN_ID(BranchN_ID),
•     .MemRW_ID(MemRW_ID),
•     .Jump_ID(Jump_ID),
•     .MemtoReg_ID(MemtoReg_ID),
•     .RegWrite_out_ID(RegWrite_out_ID),
•     .all_regs_data(all_regs_data)
• );
•     ID_reg_Ex ID_reg_Ex(
•     .clk_IDEX(clk),
•     .rst_IDEX(rst),
•     .en_IDEX(1'b1),
•     .PC_in_IDEX(PC_out_ID),
•     .Rd_addr_IDEX(Rd_addr_out_ID),
•     .Rs1_in_IDEX(Rs1_out_ID),
•     .Rs2_in_IDEX(Rs2_out_ID),
•     .Imm_in_IDEX(Imm_out_ID),
•     .ALUSrc_B_in_IDEX(ALUSrc_B_ID),
•     .ALU_control_in_IDEX(ALU_control_ID),
•     .Branch_in_IDEX(Branch_ID),
•     .BranchN_in_IDEX(BranchN_ID),
•     .MemRW_in_IDEX(MemRW_ID),
•     .Jump_in_IDEX(Jump_ID),

```

```

•      .MemtoReg_in_IDEX(MemtoReg_ID),
•      .RegWrite_in_IDEX(RegWrite_out_ID),
•      .PC_out_IDEX(PC_out_IDEX),
•      .Rd_addr_out_IDEX(Rd_addr_out_IDEX),
•      .Rs1_out_IDEX(Rs1_out_IDEX),
•      .Rs2_out_IDEX(Rs2_out_IDEX),
•      .Imm_out_IDEX(Imm_out_IDEX),
•      .ALUSrc_B_out_IDEX(ALUSrc_B_IDEX),
•      .ALU_control_out_IDEX(ALU_control_IDEX),
•      .Branch_out_IDEX(Branch_IDEX),
•      .BranchN_out_IDEX(BranchN_IDEX),
•      .MemRW_out_IDEX(MemRW_EX),
•      .Jump_out_IDEX(Jump_IDEX),
•      .MemtoReg_out_IDEX(MemtoReg_IDEX),
•      .RegWrite_out_IDEX(RegWrite_IDEX)
• );
•   Pipeline_Ex Ex(
•       .PC_in_EX(PC_out_IDEX),
•       .Rs1_in_EX(Rs1_out_IDEX),
•       .Rs2_in_EX(Rs2_out_IDEX),
•       .Imm_in_EX(Imm_out_IDEX),
•       .ALUSrc_B_in_EX(ALUSrc_B_IDEX),
•       .ALU_control_in_EX(ALU_control_IDEX),
•       .PC_out_EX(PC_out_EX),
•       .PC4_out_EX(PC4_out_EX),
•       .zero_out_EX(zero_out_EX),
•       .ALU_out_EX(ALU_out_EX),
•       .Rs2_out_EX(Rs2_out_EX)
• );
•   Ex_reg_Mem Ex_reg_Mem(
•       .clk_EXMem(clk),
•       .rst_EXMem(rst),
•       .en_EXMem(1'b1),
•       .PC_in_EXMem(PC_out_EX),
•       .PC4_in_EXMem(PC4_out_EX),
•       .Rd_addr_EXMem(Rd_addr_out_IDEX),
•       .zero_in_EXMem(zero_out_EX),
•       .ALU_in_EXMem(ALU_out_EX),
•       .Rs2_in_EXMem(Rs2_out_EX),
•       .Branch_in_EXMem(Branch_IDEX),
•       .BranchN_in_EXMem(BranchN_IDEX),
•       .MemRW_in_EXMem(MemRW_EX),
•       .Jump_in_EXMem(Jump_IDEX),
•       .MemtoReg_in_EXMem(MemtoReg_IDEX),

```

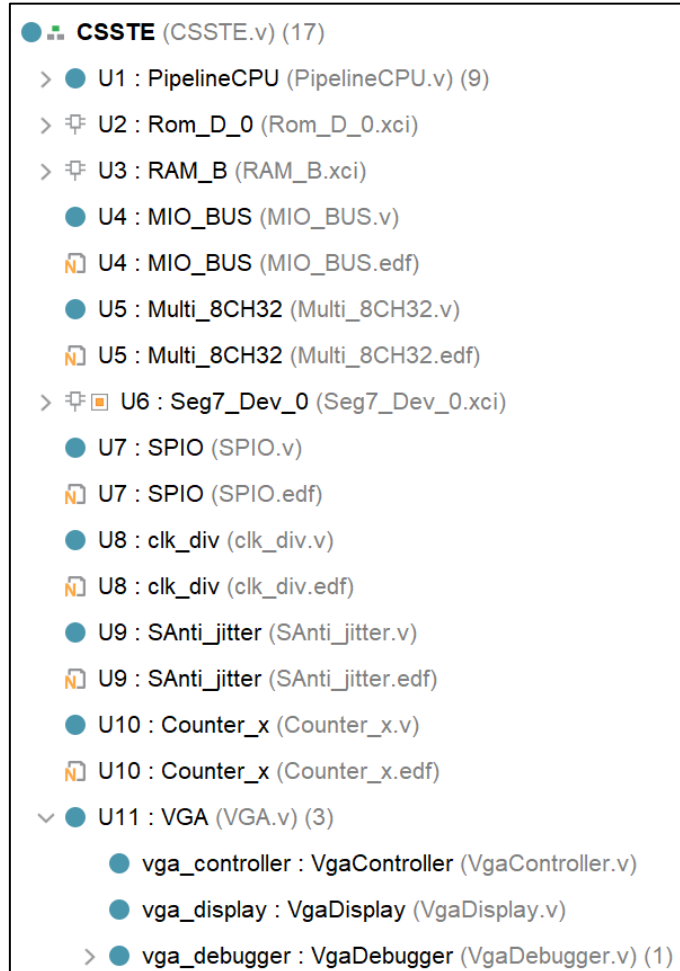
```

• .RegWrite_in_EXMem(RegWrite_IDEX),
• .PC_out_EXMem(PC_out_EXMem),
• .PC4_out_EXMem(PC4_out_EXMem),
• .Rd_addr_out_EXMem(Rd_addr_out_EXMem),
• .zero_out_EXMem(zero_out_EXMem),
• .ALU_out_EXMem(Addr_out),
• .Rs2_out_EXMem(Data_out),
• .Branch_out_EXMem(Branch_out_EXMem),
• .BranchN_out_EXMem(BranchN_out_EXMem),
• .MemRW_out_EXMem(MemRW_Mem),
• .Jump_out_EXMem(Jump_out_EXMem),
• .MemtoReg_out_EXMem(MemtoReg_out_EXMem),
• .RegWrite_out_EXMem(RegWrite_out_EXMem)
• );
• Pipeline_Mem Mem(
• .zero_in_Mem(zero_out_EXMem),
• .Branch_in_Mem(Branch_out_EXMem),
• .BranchN_in_Mem(BranchN_out_EXMem),
• .Jump_in_Mem(Jump_out_EXMem),
• .PCSrc(PCSrc) );
• Mem_reg_WB Mem_reg_WB(
• .clk_MemWB(clk),
• .rst_MemWB(rst),
• .en_MemWB(1'b1),
• .PC4_in_MemWB(PC4_out_EXMem),
• .Rd_addr_MemWB(Rd_addr_out_EXMem),
• .ALU_in_MemWB(Addr_out),
• .DMem_data_MemWB(Data_in),
• .MemtoReg_in_MemWB(MemtoReg_out_EXMem),
• .RegWrite_in_MemWB(RegWrite_out_EXMem),
• .PC4_out_MemWB(PC4_out_MemWB),
• .Rd_addr_out_MemWB(Rd_addr_out_MemWB),
• .ALU_out_MemWB(ALU_out_MemWB),
• .DMem_data_out_MemWB(DMem_data_out_MemWB),
• .MemtoReg_out_MemWB(MemtoReg_out_MemWB),
• .RegWrite_out_MemWB(RegWrite_out_MemWB)
• );
• Pipeline_WB WB(
• .PC4_in_WB(PC4_out_MemWB),
• .ALU_in_WB(ALU_out_MemWB),
• .DMem_data_WB(DMem_data_out_MemWB),
• .MemtoReg_in_WB(MemtoReg_out_MemWB),
• .Data_out_WB(Data_out_WB)
• );

```

3.2 建立流水线 SOC 工程:

Lab4 对单周期 CPU 进行了实现, 并设计了 SOC 工程 CSSTE.v 进行了相关物理验证, Lab5 也将进行物理验证, 因此需要对 CSSTE.v 进行微调 (附带整个工程结构图)。



```
module CSSTE(  
    input clk_100mhz,  
    input RSTN,  
    input [3:0] BTN_y,  
    input [15:0] SW,  
    output [3:0] Blue,  
    output [3:0] Green,  
    output [3:0] Red,  
    output HSYNC,  
    output VSYNC,  
    output [15:0] LED_out,  
    output [7:0] segment,  
    output [7:0] AN  
);
```

```

wire [3:0] BTN_OK;
wire [15:0] SW_OK;
wire rst;
wire mem_w;
wire [31:0] Cpu_data2bus ,Cpu_data4bus;
wire [31:0] addr;
wire [31:0] ram_data_out;
wire [31:0] counter_out;
wire counter0_out;
wire counter1_out;
wire counter2_out;
wire [31:0] ram_data_in;
wire [9:0] ram_addr;
wire data_ram_we;
wire GPIOf0000000_we;
wire GPIOe0000000_we;
wire counter_we;
wire [31:0] Peripheral_in;
wire [31:0] clk_div;
wire clk_CPU;
wire [1:0] counter_set;
wire [63:0] point_in;
wire [31:0]
wire [31:0]
PC_out_IF,PC_out_ID,PC_out_Ex,inst_ID,inst_IF,Data_out_WB;
wire MemRW_EX , MemRW_Mem ;
wire [1023:0] all_regs_data;

assign point_in = {clk_div,clk_div};
PipelineCPU U1(
    .clk(clk_CPU),
    .rst(rst),
    .Data_in(Cpu_data4bus),
    .PC_out_IF(PC_out_IF[31:0]),
    .inst_IF(inst_IF),
    .PC_out_ID(PC_out_ID[31:0]),
    .inst_ID(inst_ID),
    .PC_out_IDEX(PC_out_Ex),
    .MemRW_EX(MemRW_EX),
    .MemRW_Mem(MemRW_Mem),
    .Data_out(Cpu_data2bus),
    .Addr_out(addr),
    .Data_out_WB(Data_out_WB),
    .all_regs_data(all_regs_data));

```

```

Rom_D_0 U2 (
    .a(PC_out_IF[11:2]),
    .spo(inst_IF)
);

wire [31:0] RAM_B_douta;
RAM_B U3(
    .addra(ram_addr),
    .clka(~clk_100mhz),
    .dina(ram_data_in),
    .douta(RAM_B_douta), // U4 & U11
    .wea(data_ram_we)
);

MIO_BUS U4(
    .clk(clk_100mhz),
    .rst(rst),
    .BTN(BTN_OK[3:0]),
    .SW(SW_OK[15:0]),
    .mem_w(MemRW_Mem),
    .Cpu_data2bus(Cpu_data2bus[31:0]),
    .addr_bus(addr[31:0]),
    .ram_data_out(RAM_B_douta[31:0]),
    .led_out(LED_out[15:0]), //From U7
    .counter_out(counter_out[31:0]), // From U10
    .counter0_out(counter0_out),
    .counter1_out(counter1_out),
    .counter2_out(counter2_out), // all U10
    .Cpu_data4bus(Cpu_data4bus[31:0]),
    .ram_data_in(ram_data_in[31:0]),
    .ram_addr(ram_addr[9:0]),
    .data_ram_we(data_ram_we),
    .GPIOf0000000_we(GPIOf0000000_w),
    .GPIOe0000000_we(GPIOe0000000_we),
    .counter_we(counter_we),
    .Peripheral_in(Peripheral_in[31:0])
);

wire [7:0] point_out;
wire [7:0] le_out;
wire [31:0] disp_num;

Multi_8CH32 U5(
    .clk(~clk_CPU),

```

```

        .rst(rst),
        .EN(GPIOe0000000_we),
        .point_in(point_in),
        .LES(64'b0),
        .Test(SW_OK[7:5]),
        .Data0(Peripheral_in),
        .data1({2'b00,PC_out_IF[31:2]}),
        .data2(inst_IF),
        .data3(counter_out),
        .data4(addr),
        .data5(Cpu_data2bus),
        .data6(Cpu_data4bus),
        .data7(PC_out_IF),
        .point_out(point_out),
        .LE_out(le_out),
        .Disp_num(dis_num)
    );
    Seg7_Dev_0 U6(
        .disp_num(dis_num),
        .point(point_out),
        .les(le_out),
        .scan({clk_div[18],clk_div[17],clk_div[16]}),
        .AN(AN),
        .segment(segment)
    );
    SPIO U7(
        .clk(~clk_CPU),
        .rst(rst),
        .Start(clk_div[20]),
        .EN(GPIOf0000000_we),
        .P_Data(Peripheral_in),
        .counter_set(counter_set),
        .LED_out(LED_out)
    );

    clk_div U8(
        .clk(clk_100mhz),
        .rst(rst),
        .SW2(SW_OK[2]),
        .SW8(SW_OK[8]),
        .STEP(SW_OK[10]),
        .clkdiv(clk_div),
        .Clk_CPU(clk_CPU)
    );

```

```

SAnti_jitter U9(
    .clk(clk_100mhz),
    .RSTN(RSTN),
    .Key_y(BTN_y),
    .SW(SW[15:0]),
    .BTN_OK(BTN_OK[3:0]),
    .SW_OK(SW_OK[15:0]),
    .rst(rst)
);

Counter_x U10(
    .clk(~clk_CPU),
    .rst(rst),
    .clk0(clk_div[6]),
    .clk1(clk_div[9]),
    .clk2(clk_div[11]),
    .counter0_OUT(counter0_out),
    .counter1_OUT(counter1_out),
    .counter2_OUT(counter2_out),
    .counter_we(counter_we),
    .counter_out(counter_out),
    .counter_val(Peripheral_in),
    .counter_ch(counter_set)
);

VGA U11(
    .clk_25m(clk_div[1]),
    .clk_100m(clk_100mhz),
    .rst(rst),
    .PC_IF(PC_out_IF[31:0]),
    .PC_ID(PC_out_ID[31:0]),
    .inst_IF(inst_IF),
    .inst_ID(inst_ID),
    .PC_Ex(PC_out_Ex),
    .MemRW_Ex(MemRW_EX),
    .MemRW_Mem(MemRW_Mem),
    .Data_out(Cpu_data2bus),
    .Addr_out(addr),
    .Data_out_WB(Data_out_WB),
    .hs(HSYNC),
    .vs(VSYNC),
    .vga_r(Red),
    .vga_g(Green),
    .vga_b(Blue),
    .all_regs_data(all_regs_data) );

```

```
endmodule
```


四、实验结果分析

```
PipelineCPU U1(
    .clk(clk_CPU),
    .rst(rst),
    .Data_in(Cpu_data4bus),
    .PC_out_IF(PC_out_IF[31:0]),
    .inst_IF(inst_IF),
    .PC_out_ID(PC_out_ID[31:0]),
    .inst_ID(inst_ID),
    .PC_out_IDEX(PC_out_Ex),
    .MemRW_EX(MemRW_EX),
    .MemRW_Mem(MemRW_Mem),
    .Data_out(Cpu_data2bus),
    .Addr_out(addr),
    .Data_out_WB(Data_out_WB),
    .all_regs_data(all_regs_data)
);

Rom_D_0 U2 (
    .a(PC_out_IF[11:2]),
    .spo(inst_IF)
);

wire [31:0] RAM_B_douta;

RAM_B U3(
    .addra(ram_addr),
    .clka(~clk_100mhz),
    .dina(ram_data_in),
    .douta(RAM_B_douta), // U4 & U11
    .wea(data_ram_we)
);

MIO_BUS U4(
    .clk(clk_100mhz),
    .rst(rst),
    .BTN(BTN_OK[3:0]),
    .SW(SW_OK[15:0]),
    .mem_w(MemRW_Mem),
    .Cpu_data2bus(Cpu_data2bus[31:0]),
    .addr_bus(addr[31:0]),
    .ram_data_out(RAM_B_douta[31:0]),
    .led_out(LED_out[15:0]), //From U7
    .counter_out(counter_out[31:0]), // From U10
    .counter0_out(counter0_out),
    .counter1_out(counter1_out),
    .counter2_out(counter2_out), // all U10
    .Cpu_data4bus(Cpu_data4bus[31:0]),
```

Lab5 由于设计的 Pipeline_CPU 输入需要来自 MIO_BUS (.edf 文件) 的信号 Cpu_data4bus、Cpu_data2bus 等，但对 PipelineCPU 进行仿真平台的构建时可以将 Cpu_data4bus 直接替换为 RAM_B_douta 进行连续指令仿真(Lab5-3 时仿真)。

同时为了实验报告内容尽量不重复，物理验证统一在 Lab5-2~3 自主实现流水线各阶段代码并且替换.edf 文件后再进行物理验证。

五、讨论与心得

单纯的连线操作，但由于流水线 CPU 较复杂且提供的文件中有较多小错误，需要仔细检查，不然后期实验 debug 很痛苦。